Royaume du Maroc

Compte rendu de TP

TP 7 : NumPy

Module : Programmation en Python

Réalisé par : AKEBBAD MOHCINE

Filière / classe : MGSI1

Encadré par : Pr. JABRANE Mourad

Année universitaire : 2026–2027

Table des matières

Introduction **2**

1 Exercice 1 : Création et propriétés d'un tableau **2**

2 Exercice 2 : Tableau rempli de zéros **3**

3 Exercice 3 : Conversion du type **4**

4 Exercice 4 : Valeurs uniformément réparties **5**

5 Exercice 5 : Matrice identité et diagonale **6**

6 Exercice 6 : Redimensionnement d'un tableau **7**

7 Exercice 7 : Extraction d'une sous-matrice **8**

8 Exercice 8 : Filtrage de valeurs aléatoires **9**

9 Exercice 9 : Opérations élément par élément **10**

10 Exercice 10 : Somme par colonne et moyenne par ligne **11**

11 Exercice 11 : Moyenne en ignorant les valeurs manquantes **12**

12 Exercice 12 : Produit matriciel **13**

13 Exercice 13 : Tri et indices de tri **14**

14 Exercice 14 : Remplacement des valeurs négatives **15**

15 Exercice 15 : Jeu de données pour une régression linéaire **16**

Conclusion **17**

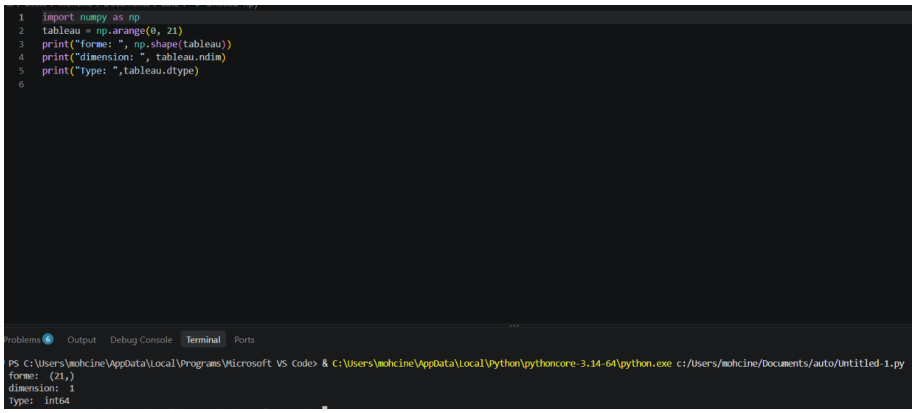
Introduction

Ce travail pratique a pour objectif de se familiariser avec la bibliothèque NumPy de Python et ses principales opérations sur les tableaux et les matrices. Les captures du code Python et de son exécution illustrent le travail réalisé pour chaque exercice.

Consigne du TP : aucune boucle explicite (`for`, `while`) n'est autorisée, sauf indication contraire.

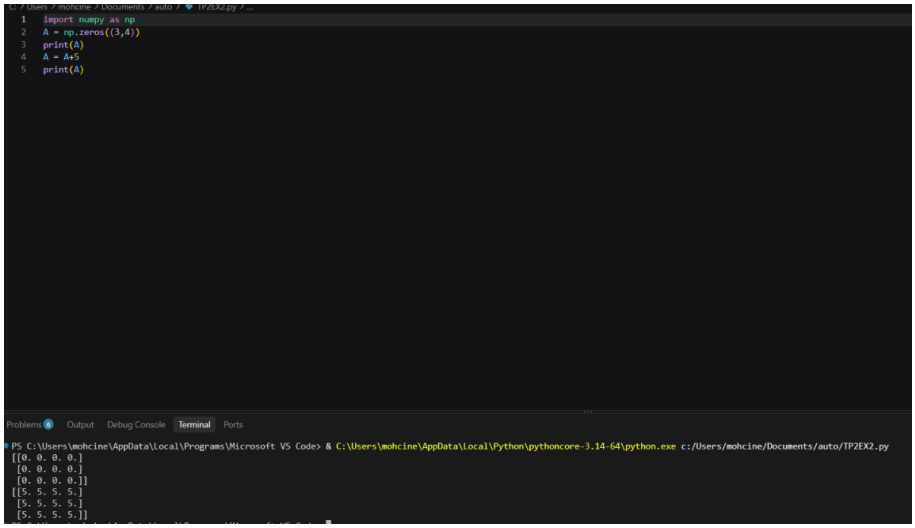
1 Exercice 1 : Création et propriétés d'un tableau

Créer un tableau NumPy contenant les entiers de 0 à 20 inclus, puis afficher sa forme, sa dimension et son type.



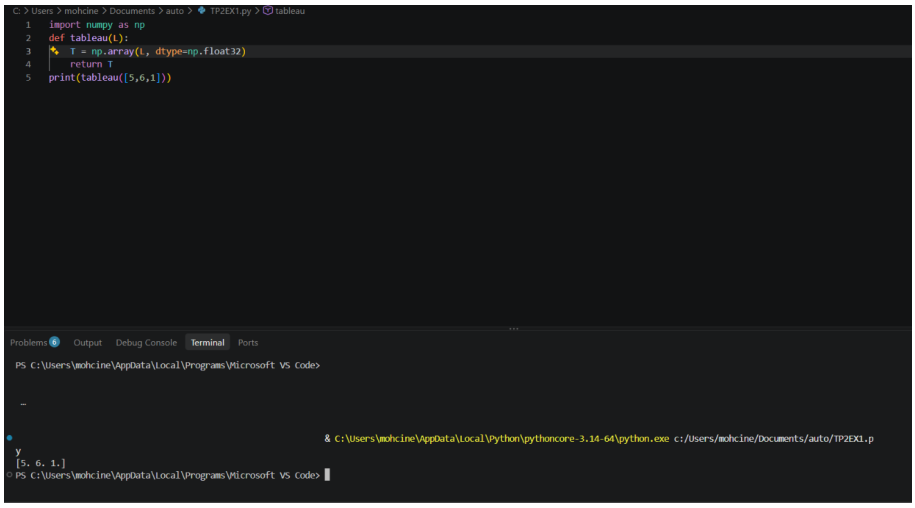
2 Exercice 2 : Tableau rempli de zéros

Créer un tableau à deux dimensions de taille 3×4, rempli de zéros, puis remplacer tous les zéros par la valeur 5.



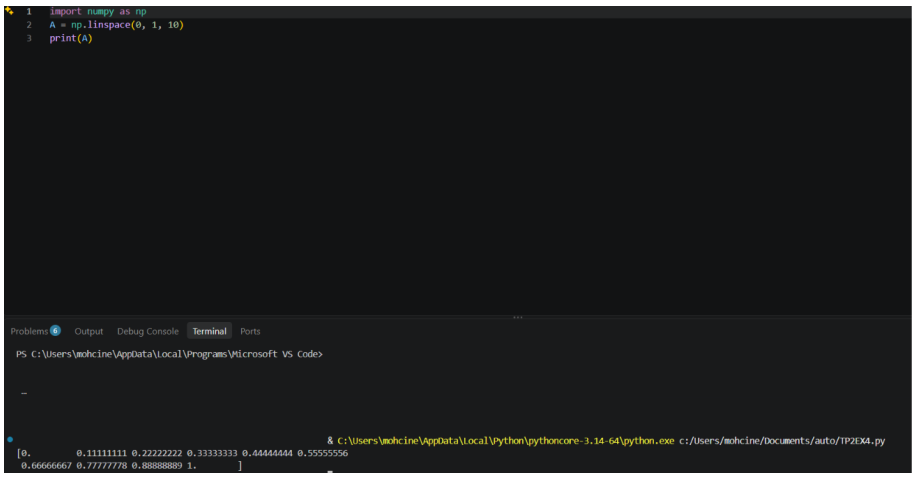
3 Exercice 3 : Conversion du type

À partir d'une liste Python contenant des nombres entiers, créer un tableau NumPy et convertir son type en nombres réels (`float`).



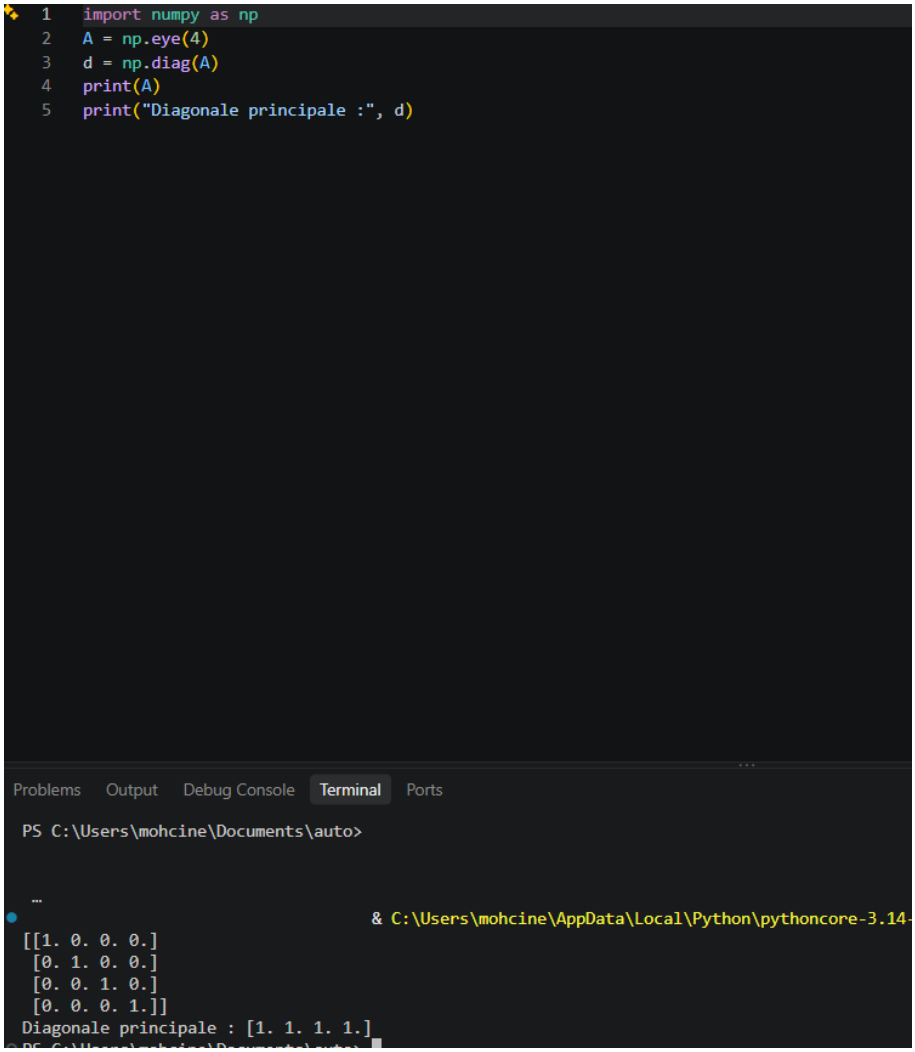
4 Exercice 4 : Valeurs uniformément réparties

Créer un tableau contenant 10 valeurs uniformément réparties entre 0 et 1.



5 Exercice 5 : Matrice identité et diagonale

Créer une matrice identité de taille 4×4, puis extraire sa diagonale principale.



6 Exercice 6 : Redimensionnement d'un tableau

Créer un tableau unidimensionnel de taille 12, puis le transformer en une matrice de taille 3×4.

```python
import numpy as np
a = np.arange(12)
b = a.reshape(3, 4)
print(a)
print(b)
```

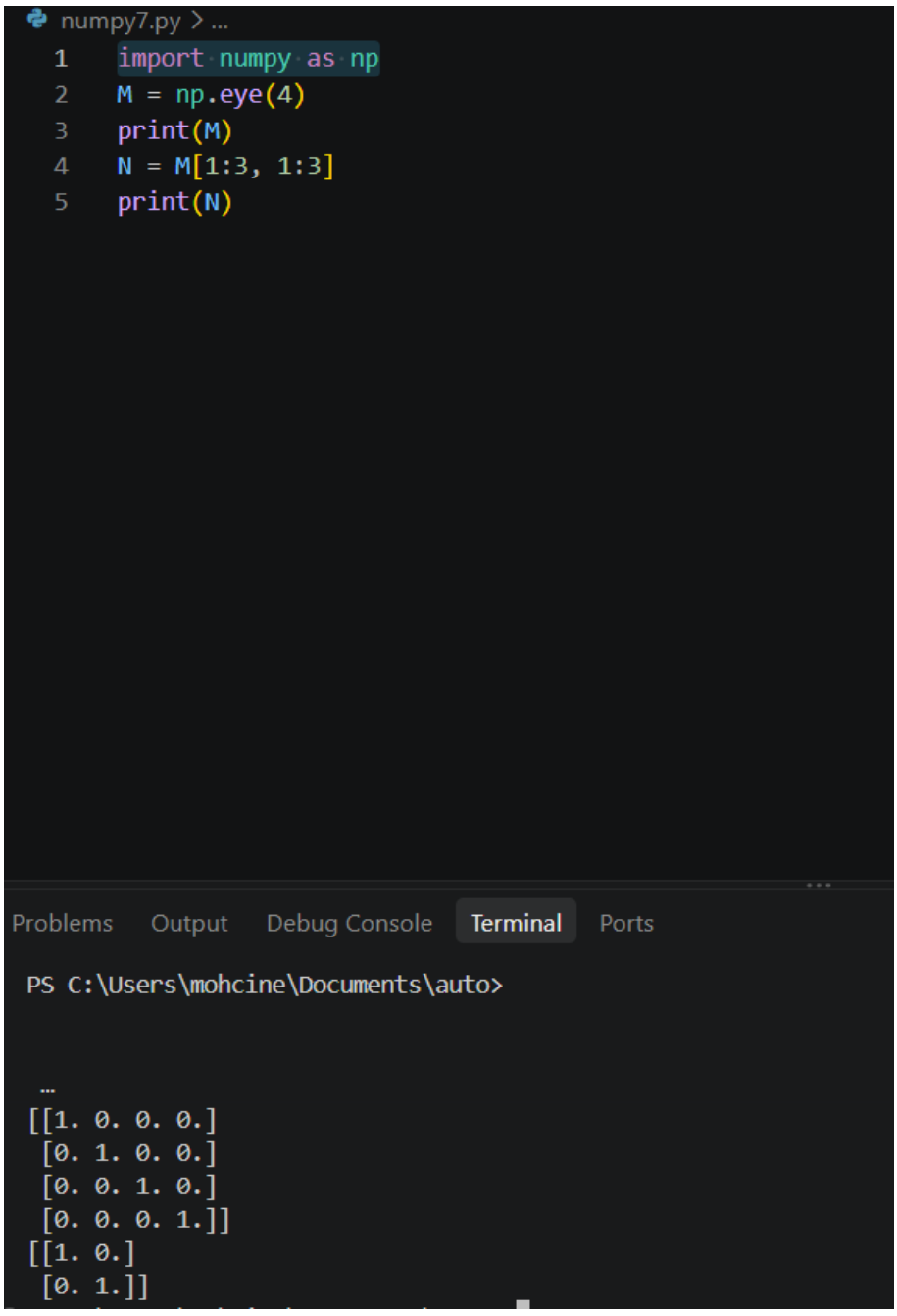
```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\mohcine\Documents\auto>

                                & C:\Users\mohcine\AppData\Local\Python\
.exe c:/Users/mohcine/Documents/auto/numpy6.py
[ 0  1  2  3  4  5  6  7  8  9 10 11]
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

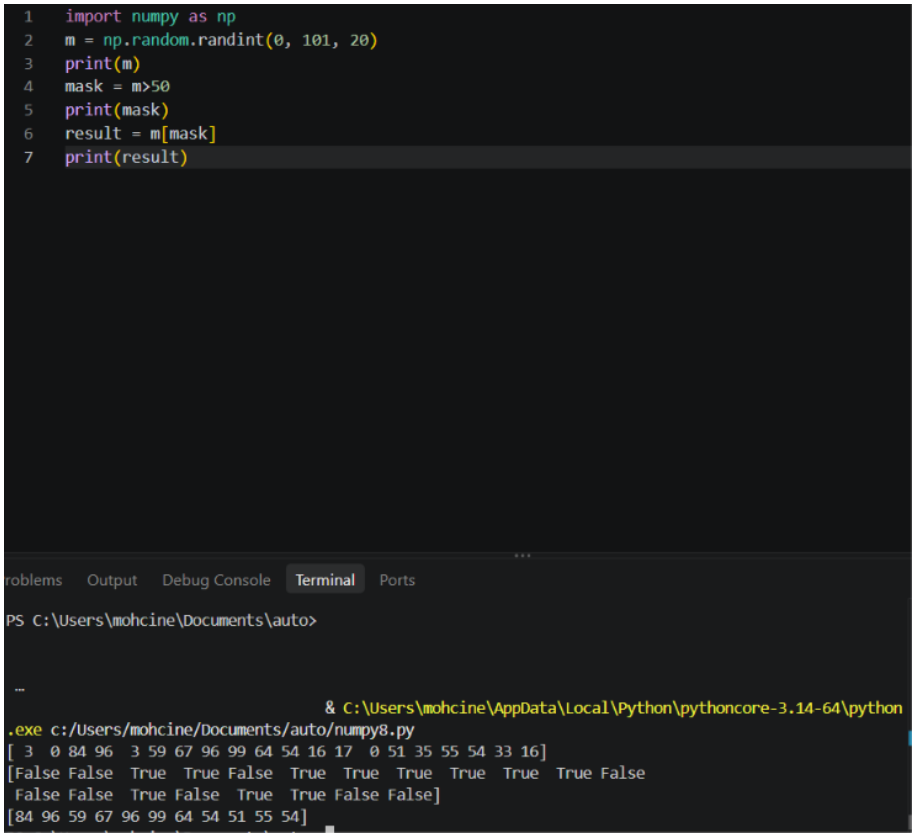
7 Exercice 7 : Extraction d'une sous-matrice

À partir d'une matrice 4×4, extraire la sous-matrice centrale de taille 2×2.



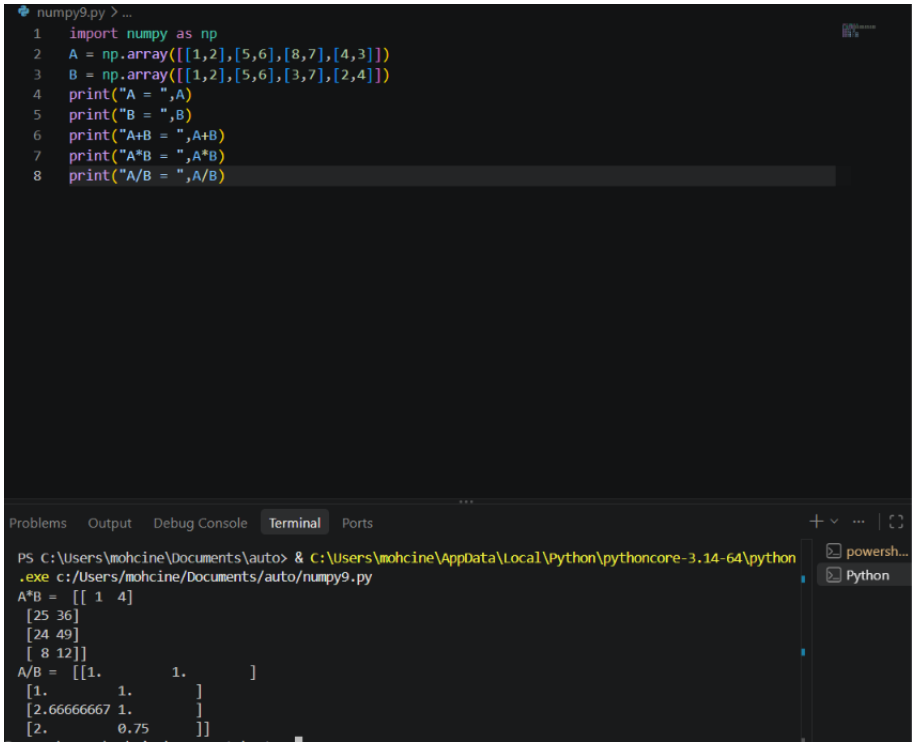
8 Exercice 8 : Filtrage de valeurs aléatoires

Créer un tableau de 20 entiers aléatoires compris entre 0 et 100, puis sélectionner uniquement les valeurs strictement supérieures à 50.



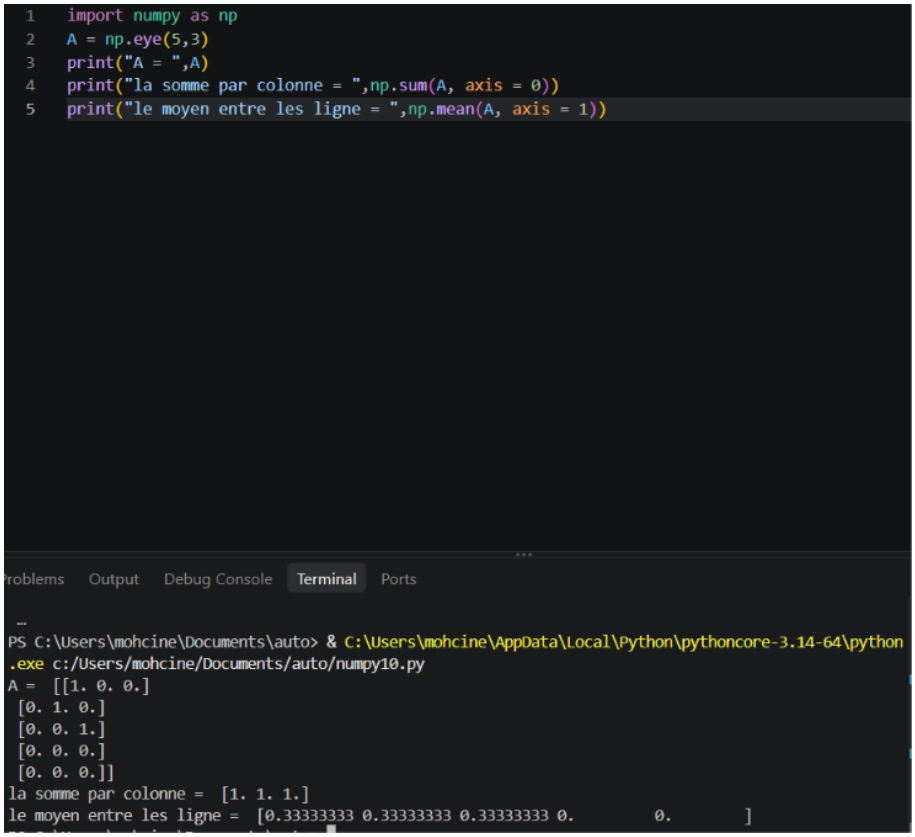
9 Exercice 9 : Opérations élément par élément

Créer deux tableaux NumPy de même taille et effectuer les opérations suivantes : addition, soustraction, multiplication et division élément par élément.



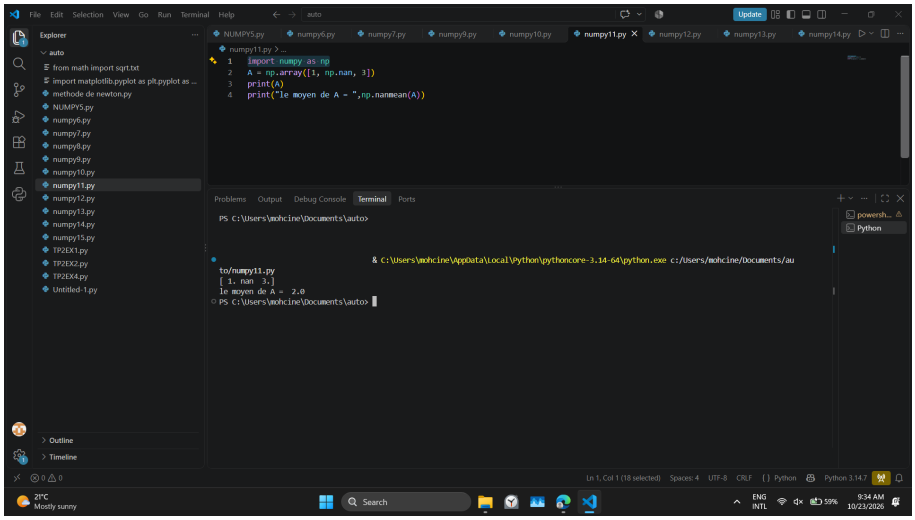
10 Exercice 10 : Somme par colonne et moyenne par ligne

Créer une matrice de taille 5×3, puis calculer la somme de chaque colonne et la moyenne de chaque ligne.



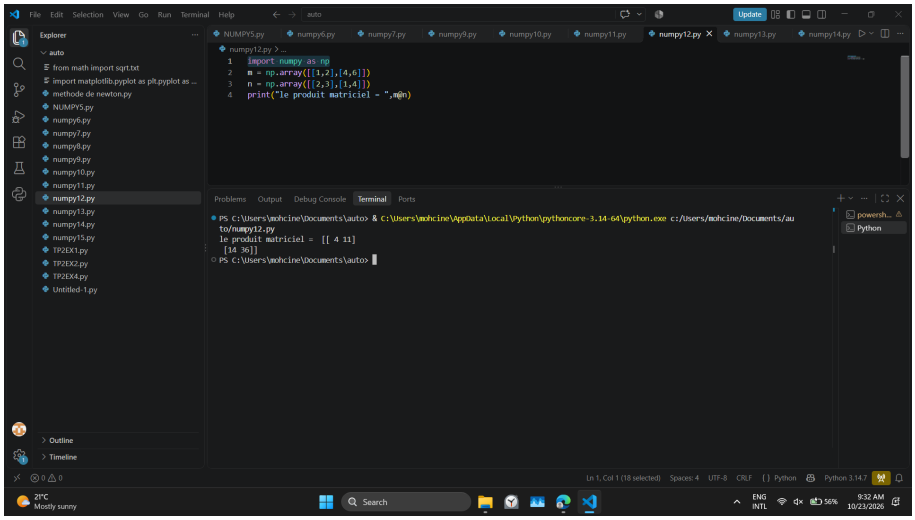
11 Exercice 11 : Moyenne en ignorant les valeurs manquantes

À partir d'un tableau contenant des valeurs manquantes représentées par `np.nan`, calculer la moyenne en ignorant ces valeurs.



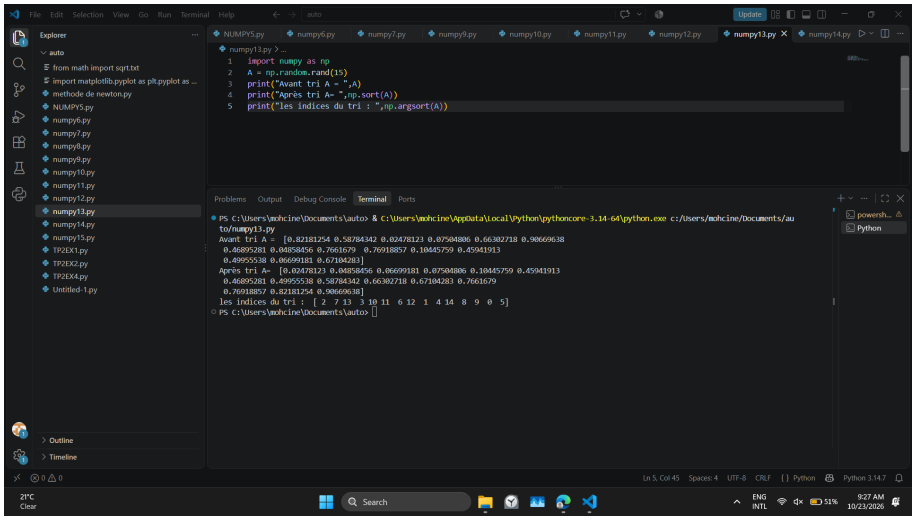
12 Exercice 12 : Produit matriciel

Créer deux matrices compatibles et calculer leur produit matriciel en utilisant l'opérateur `@`.



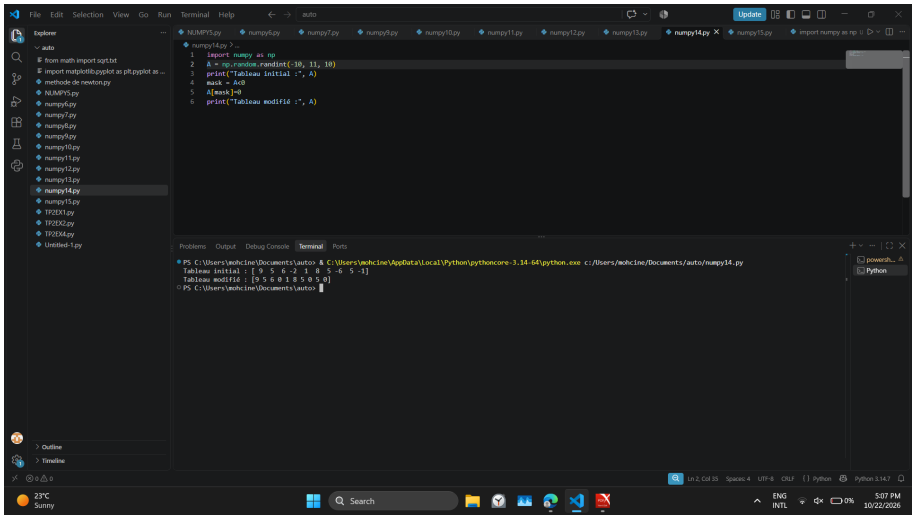
13 Exercice 13 : Tri et indices de tri

Générer un tableau aléatoire de taille 15, trier ses valeurs et afficher les indices correspondant à l'ordre de tri.



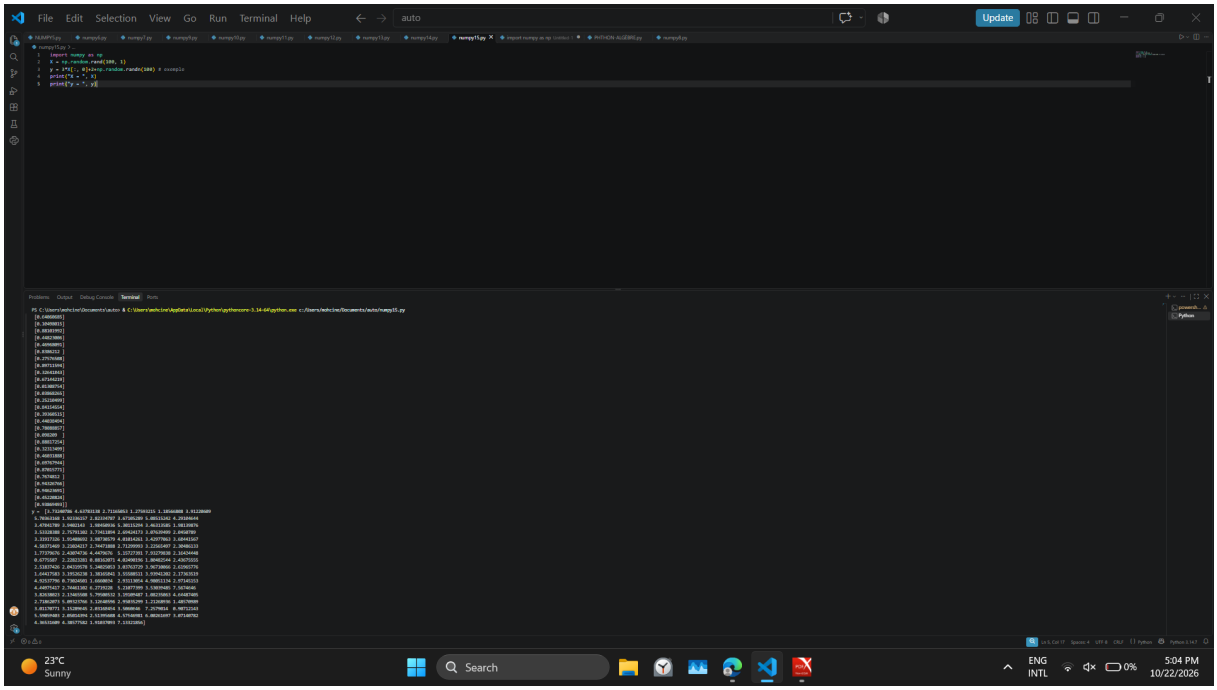
14 Exercice 14 : Remplacement des valeurs négatives

Créer un tableau contenant 10 valeurs aléatoires, puis remplacer toutes les valeurs négatives par 0.



15 Exercice 15 : Jeu de données pour une régression linéaire

Générer un jeu de données simulé (X, y) pour une régression linéaire simple, où X est une matrice de taille 100×1 et y un vecteur de taille 100.



Conclusion

Ce TP a permis d'aborder plusieurs fonctionnalités essentielles de NumPy : création et manipulation de tableaux, indexation, opérations élémentaires, calculs matriciels, traitement des valeurs manquantes, tri et génération de données aléatoires. Les captures d'écran du code et de son exécution permettent de documenter le travail réalisé pour chaque exercice.